

JavaScript Promises – Complete Theory Notes

1. Introduction to Promises

A **Promise** in JavaScript is an object that represents the **eventual completion** or **failure** of an asynchronous operation.

It allows us to write cleaner asynchronous code without falling into “callback hell”.

A Promise has three states:

1. **Pending** – Initial state, operation not completed
 2. **Fulfilled** – Operation completed successfully
 3. **Rejected** – Operation failed
-

2. Creating a Promise

javascript

 Copy code

```
let promise = new Promise((resolve, reject) => {  
  // Async task  
  if (success) {  
    resolve("Task completed");  
  } else {  
    reject("Error occurred");  
  }  
});
```

- `resolve()` → moves the promise to **fulfilled**
 - `reject()` → moves the promise to **rejected**
-

3. Consuming a Promise

We use:

✓ **.then()**

Runs when a promise resolves.

javascript

 Copy code

```
promise.then((value) => {  
    console.log(value);  
});
```

✓ .catch()

Runs when a promise is rejected.

javascript

 Copy code

```
promise.catch((error) => {  
    console.log(error);  
});
```

✓ .finally()

Runs whether the promise resolves or rejects.

javascript

 Copy code

```
promise.finally(() => {  
    console.log("Always runs");  
});
```

4. Promise Chaining

Each `.then()` returns a new promise — allowing us to chain multiple operations in order.

javascript

 Copy code

```
promise  
    .then((data) => {  
        console.log("Step 1:", data);  
        return 20;  
    })  
    .then((value) => {  
        console.log("Step 2:", value);  
        return 30;  
    })  
    .then((value) => {
```

```
        console.log("Step 3:", value);
    })
    .catch((err) => {
        console.log("Caught:", err);
    })
    .finally(() => {
        console.log("Finished");
    });
```

Why chaining?

It avoids nested callbacks and makes logic easier to read.

5. Promise.all()

`Promise.all()` is used to run multiple promises in parallel.

javascript

 Copy code

```
Promise.all([p1, p2, p3])
    .then((results) => {
        console.log(results);
    })
    .catch((err) => {
        console.log(err); // Runs if ANY promise rejects
    });
```

Behavior:

- If **all** promises resolve → returns an array of results
 - If **any one** rejects → entire `Promise.all()` goes to `catch()`
-

6. Promise.allSettled()

Runs all promises and returns results even if some fail.

javascript

 Copy code

```
Promise.allSettled([p1, p2, p3])
```

```
.then(console.log);
```

Output example:

lua

 Copy code

```
[  
  { status: "fulfilled", value: ... },  
  { status: "rejected", reason: ... }  
]
```

7. Promise.race()

Returns the result of the **first promise** to settle (resolve or reject).

javascript

 Copy code

```
Promise.race([p1, p2])  
  .then(console.log)  
  .catch(console.error);
```

8. Promise.any()

Returns the **first fulfilled promise**.

Ignores rejected ones unless *all* fail.

javascript

 Copy code

```
Promise.any([p1, p2, p3])  
  .then(console.log)  
  .catch(console.error); // Only if all fail
```

9. Using Promises with setTimeout

Example:

javascript

 Copy code

```
let promise = new Promise((resolve) => {
  setTimeout(() => {
    resolve("Done!");
  }, 2000);
});
```

10. Why Promises?

- ✓ Avoid callback hell
- ✓ Better error handling
- ✓ Cleaner async code
- ✓ Supports chaining
- ✓ Works well with `async/await`

11. Promises vs Callbacks

Feature	Callback	Promise
Syntax	Nested	Flat & clean
Error Handling	Difficult	Easy with <code>.catch()</code>
Chaining	Hard	Simple
Readability	Low	High

12. Promises with `async/await`

Async/await is built on top of Promises.